# Si-Five Report: Systolic Array Emulation for Matrix Multiplication

Pranav Suresh

October 2026

**Abstract**

This report details the implementation of a systolic array emulator designed to execute matrix multiplication on a bare-metal Si-Five RISC-V platform. The primary objective is to evaluate and compare the computational speedups offered by two distinct systolic dataflows: the traditional Weight-Stationary (WS) dataflow and the optimized Diagonal-Input Permutated (DiP) dataflow. By benchmarking these hardware-emulated accelerators against a scalar CPU baseline, this study highlights the latency trade-offs, memory management strategies in resource-constrained environments, and bare-metal verification techniques.

## 1 Introduction

Matrix multiplication is a foundational operation for high-performance computing and machine learning. Hardware accelerators, specifically systolic arrays, are heavily relied upon to speed up these workloads. To validate novel architectural dataflows before committing them to silicon, software emulators running on embedded processing cores serve as an essential intermediary step. This project implements such an emulator on a Si-Five board, analyzing how matrix tiling, memory constraints, and dataflow topologies impact overall execution speed.

## 2 Architecture Overview and Memory Constraints

The system models the multiplication of two matrices, $A$ ($M \times N$) and $B$ ($N \times K$), resulting in an output matrix $C$ ($M \times K$). For this study, the global dimensions are defined as `M_DIM`, `N_DIM`, and `K_DIM`. Working within a bare-metal Si-Five environment introduces strict hardware constraints, the most prominent being the 16 KB SRAM limit. To accommodate this limit, the system is calibrated to handle maximum matrix sizes of $24 \times 24$. Three matrices of this size (using 32-bit floating-point numbers) consume approximately 6.9 KB, comfortably fitting inside the available memory. Furthermore, dynamic memory allocation or large local arrays heavily risk stack overflows in a bare-metal environment lacking a robust operating system. To mitigate this, the full working matrices and execution buffers are explicitly allocated in the global scope (the `.bss` segment). This static allocation strategy guarantees memory safety and predictable addressing during execution.

## 3 Workload Scheduling and Tiling

Because systolic arrays have fixed physical dimensions, large matrices must be partitioned into smaller, hardware-compatible chunks. This is managed by a dedicated software scheduler, `matmul_hwa_feeder`. The hardware array size is defined by `BASE_LIMIT` ($8 \times 8$). The scheduler iterates over the full $24 \times 24$ matrices and breaks them down into $8 \times 8$ tasks, creating a sequential queue of `HwaJob` structures. Each job pairs a specific tile of Matrix $A$ with a corresponding

tile of Matrix $B$. To maintain robustness, the scheduler implements boundary checks to handle "ragged edges"—cases where the global matrix dimensions are not perfectly divisible by the tile size. Once a tile's coordinates and boundaries are determined, the `extract_tile` function fetches the data from the main memory. If a tile crosses the boundary of the global matrix, `extract_tile` automatically applies zero-padding to ensure the hardware emulator is always fed a complete, valid $8 \times 8$ data block.

# 4 Hardware Dataflow Implementations

The core of the emulator tests two distinct systolic array dataflows. Both assume an $8 \times 8$ grid of Processing Elements (PEs), but they route data very differently.

## 4.1 Diagonal-Input Permutated (DiP) Dataflow

The DiP dataflow is a highly optimized methodology designed to eliminate synchronization delays.

- **Permutation:** Prior to computation, Matrix $B$ (the weights) is reshaped using a `permute` function, which applies a column-by-column cyclic shift.
- **Data Movement:** Matrix $A$ is fed into the array in a diagonal stream, with the leftmost row wrapping around to the rightmost column.
- **Latency:** Because the data is pre-aligned and fed diagonally, all PEs perform useful multiply-accumulate (MAC) operations immediately. Consequently, an $8 \times 8$ chunk finishes computation in exactly $N = 8$ cycles, offering maximum hardware utilization.

## 4.2 Weight-Stationary (WS) Dataflow

The Weight-Stationary dataflow is a traditional approach used in commercial accelerators like the Google TPU.

- **Stationary Weights:** The fundamental difference in WS is that the elements of Matrix $B$ are loaded into the PEs and remain entirely stationary.
- **Data Movement:** Matrix $A$ flows horizontally across the array (left to right), while the computed partial sums cascade vertically (top to bottom).
- **Latency:** To ensure that elements of $A$ meet the correct elements of $B$ at the correct time, the input rows must be staggered. Row $i$ must be delayed by $i$ clock cycles using input FIFOs. Similarly, the outputs must drain through output FIFOs. This spatial staggering drastically slows down the computation speed of a single tile, requiring a fill, compute, and drain phase that costs exactly $3N$ cycles. For an $8 \times 8$ tile, this results in $3 \times 8 = 24$ cycles—three times slower than the DiP method at the tile level.

# 5 Verification and Output Reporting

To ensure the hardware emulators produce mathematically correct results, their outputs are rigorously compared against a standard scalar CPU matrix multiplication (`full_cpu_matmul`). As the hardware processes each $8 \times 8$ job, an `accumulate_tile` function maps and adds the partial hardware results back into their proper coordinates in the full output matrix. Once all jobs are complete, the verification routine runs. Because the system utilizes floating-point arithmetic, hardware-level summation order can cause minor precision deviations. Therefore, a strict equality check is insufficient. Instead, the verification script utilizes an absolute difference

threshold ($> 1e-5$) backed by a relative epsilon constraint ($1e-4$) to determine if the hardware results fall within an acceptable tolerance. Debugging bare-metal software is challenging, particularly because the target board lacks OpenOCD support for live hardware debugging. To solve this, a volatile integer, `VERIFICATION_STATUS`, is utilized. It cycles through predefined states (`-1` for running, `0` for failed, and `1` for passed), allowing a GDB debugger to peek at the memory address and confirm the program state. Finally, a `report_verification` function serializes the exact tolerance data and success metrics, transmitting them back to the host via UART.

# 6 Conclusion

The implementation of this emulator successfully highlights the computational disparity between systolic dataflows on a RISC-V SoC. While the Weight-Stationary array requires extensive FIFO overhead resulting in a 24-cycle latency per block, the Diagonal-Input Permutated (DiP) dataflow circumvents these synchronization penalties, finishing the same workload in just 8 cycles. The memory constraints of the Si-Five board were safely navigated through static memory allocation and robust matrix tiling, creating a reliable testbed for future hardware acceleration research.